

Deploying Django as a Silent Windows Service

A Professional Deployment Guide for Local Networks & Windows 11 Environments

This documentation details the process of registering a local Python Django web application to run silently in the background as a native **Windows Service** on Windows 11. By implementing this architecture, the application will boot automatically when the machine starts up (before user login) and will automatically restart if a system crash or reboot occurs.

Why Background Services?

Unlike running `manage.py runserver` in an active terminal, a Windows Service operates silently inside session 0, requires no user action to start, consumes fewer graphical resources, and recovers dynamically from unexpected system crashes.

System Architecture Overview

The deployed service operates using the following structural components:

- **WSGI Application Server:** Waitress (a production-grade, multi-threaded pure-Python server designed specifically for high reliability on Windows environments).
- **Service Manager:** NSSM (the Non-Sucking Service Manager), which translates our Python execution script into a Windows native system process.
- **Virtual Environment:** Isolation of dependencies to ensure system-wide Python changes do not break the local web service.

Step 1: Install & Set Up the Production Web Server

Django's built-in development server (`runserver`) is not designed for continuous background execution or high-availability environments. We will utilize **Waitress**.

1. Open your command prompt or terminal, activate your virtual environment, and install Waitress:

```
pip install waitress
```

2. In your main Django project directory (the folder containing `manage.py`), create a new file named `wsgi_server.py` and paste the following Python implementation:

```
from waitress import serve
from your_project_name.wsgi import application # Replace with your actual project
directory name
```

```
if __name__ == '__main__':
    print("Starting Django via Waitress on http://0.0.0.0:8000")
    serve(application, host='0.0.0.0', port=8000, threads=6)
```

Step 2: Obtain and Position NSSM

NSSM registers Python scripts as reliable Windows system processes.

1. Download the latest release of NSSM from the official distribution: <https://nssm.cc/download>.
2. Extract the downloaded archive.
3. Locate the 64-bit binary at `ssm-2.x-xxx\win64 ssm.exe`.
4. Copy and paste `nssm.exe` directly into your main Django project folder alongside `manage.py` and your newly created `wsgi_server.py`.

Step 3: Register the Service

To configure the Windows Service, we must invoke administrative permissions.

1. Type **cmd** or **Command Prompt** in your Windows 11 search bar.
2. Right-click the application and select **Run as Administrator**.
3. Navigate to your project folder where NSSM and your files reside:

```
cd C:\path o\your\django\project
```

4. Execute the install GUI command:

```
nssm install DjangoLocalApp
```

5. An configuration user interface will appear. Populate the fields with your absolute environment paths:

Tab Name	Field	Value / Action
Application	Path	Path to your virtual environment's Python executable (e.g., <code>C:\path o env\Scripts\python.exe</code>)
Application	Startup directory	Your base Django project directory (e.g., <code>C:\path o\your\django\project</code>)
Application	Arguments	<code>wsgi_server.py</code>

Tab Name	Field	Value / Action
Details	Startup type	Set to Automatic (Delayed Start) to allow hardware and database engines to load first.

1. Click **Install service**. You will receive a confirmation message stating that the service was installed successfully.

Step 4: Control and Verify the Service

Use the administrative Command Prompt to boot the newly configured service:

```
net start DjangoLocalApp
```

Operational Controls Cheat Sheet

- **Stop the app:** `net stop DjangoLocalApp`
- **Start the app:** `net start DjangoLocalApp`
- **Graphical Management:** Press `Win + R`, type `services.msc`, and locate **DjangoLocalApp**.
- **Uninstall/Remove service:** Run `nssm remove DjangoLocalApp` inside an elevated CMD.

Step 5: Accessing the Application on a Local Network

Because Waitress is listening on all network interfaces (`0.0.0.0`), other local devices on your Wi-Fi or local area network can access the application. To ensure uninterrupted access after system reboots, choose one of these routing options:

Option A: Accessing via Local Computer Name (Recommended)

Windows machine names are resolved dynamically on local networks. Users can connect without needing to know your IP address:

1. Identify your computer name on Windows 11 by navigating to **Settings > System > About**.
2. If your computer's name is `WorkStation-X`, network users can visit:

```
http://WorkStation-X:8000
```

Option B: Configuring a Static IP

If utilizing specific IP configurations, lock your laptop's IP address to prevent routers from reassigning it on boot:

1. Go to **Settings > Network & internet > Wi-Fi** (or Ethernet) and click your network properties.
2. Find **IP assignment**, click **Edit**, select **Manual**, toggle **IPv4**, and enter a permanent IP address.